FIRST-YEAR B.TECH • PL 1 C PROGRAMMING

Decision Making & Looping Statements



Learning outcomes

Decision Making Statements:

Simple if, The if… else Statement, Nesting of if… else statements, The else if ladder,

The Switch Statement.

Loping Statements:

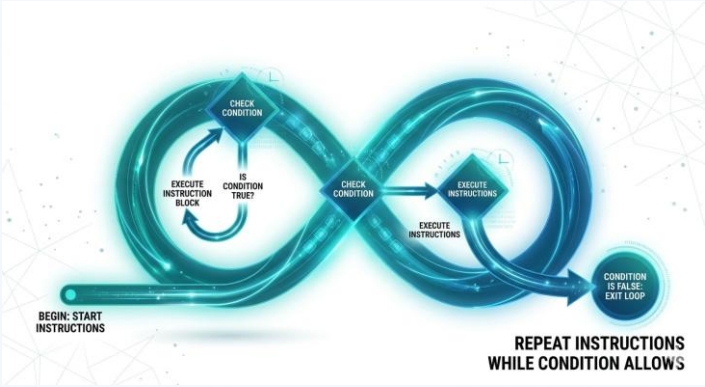
The While Statement, The Do Statement, The FOR Statement. Jumps in Loop (breakand continue statement)

- Use conditions to make decisions in a C program.
- Understand if, if–else, nested if, else-if ladder and switch.
- Repeat tasks using while, do–while and for loops.
- Control loops using break and continue.
- Compile and test small C programs in Dev-C++.

Decision making vs looping



- Decision making chooses which path to follow.
- Looping repeats instructions while a condition allows.
- break means stop the nearest loop or switch immediately.
- continue means skip the rest of the current loop iteration.
- Use indentation and braces to make code easier to understand.



Simple if — flowchart and algorithm



ALGORITHM

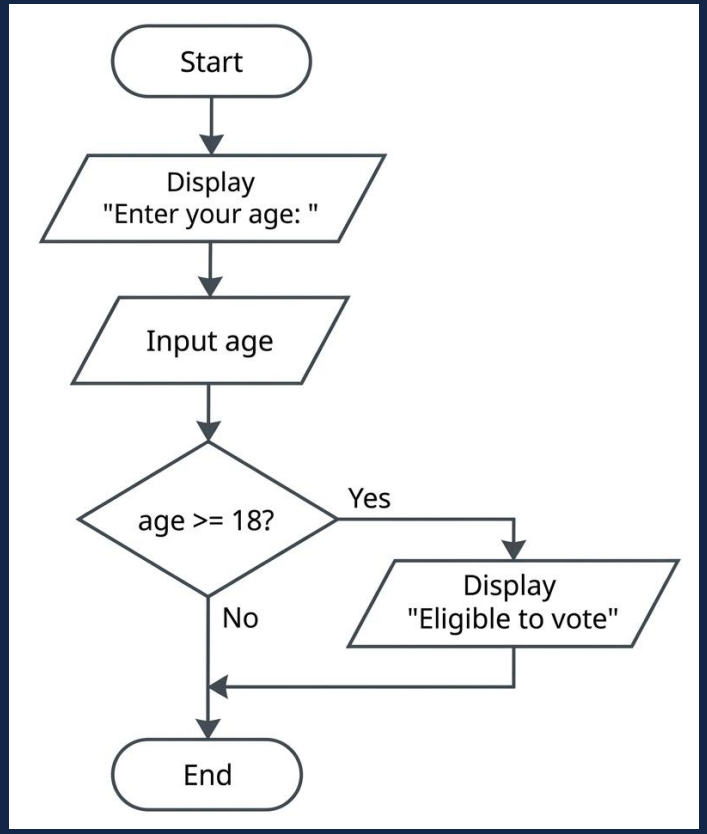
1. Start.
2. Declare and read a value.
3. Check the condition.
4. If it is true, display the message.
5. Stop.

Program 1 — Simple if: Positive number

```c
#include <stdio.h>

int main(void)
{
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18)
    {
        printf("Eligible to vote");
    }
    return 0;
}
```



KEY IDEA

- Runs the message only when num > 0.
- If input is zero or negative, no message is printed.

SAMPLE OUTPUT

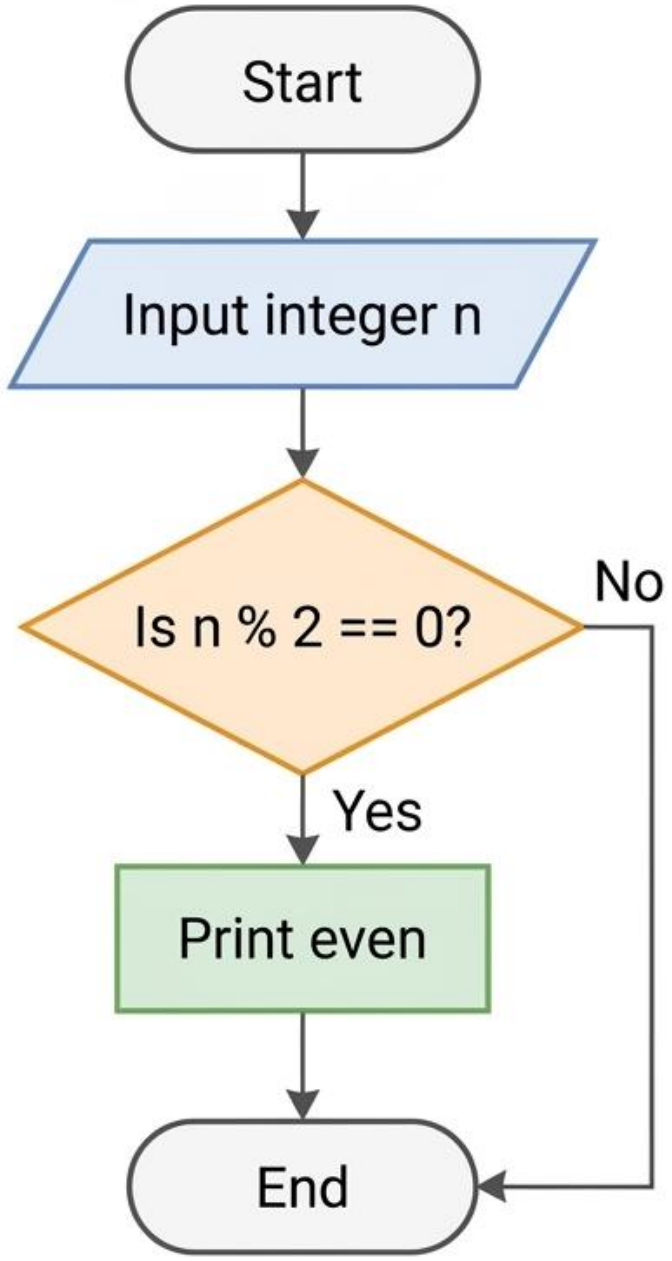
```
Enter a number: 8
Positive number
```

Program 2 — Simple if: Voting eligibility

```c
#include <stdio.h>

int main(void)
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0)
    {
        printf("Positive number");
    }
    return 0;
}
```



if–else — flowchart and algorithm

ALGORITHM

1. Start.
2. Read the input.
3. Check the condition.
4. Execute the true branch or the false branch.
5. Stop.

Program 3 — if–else: Even or odd

```c
#include <stdio.h>

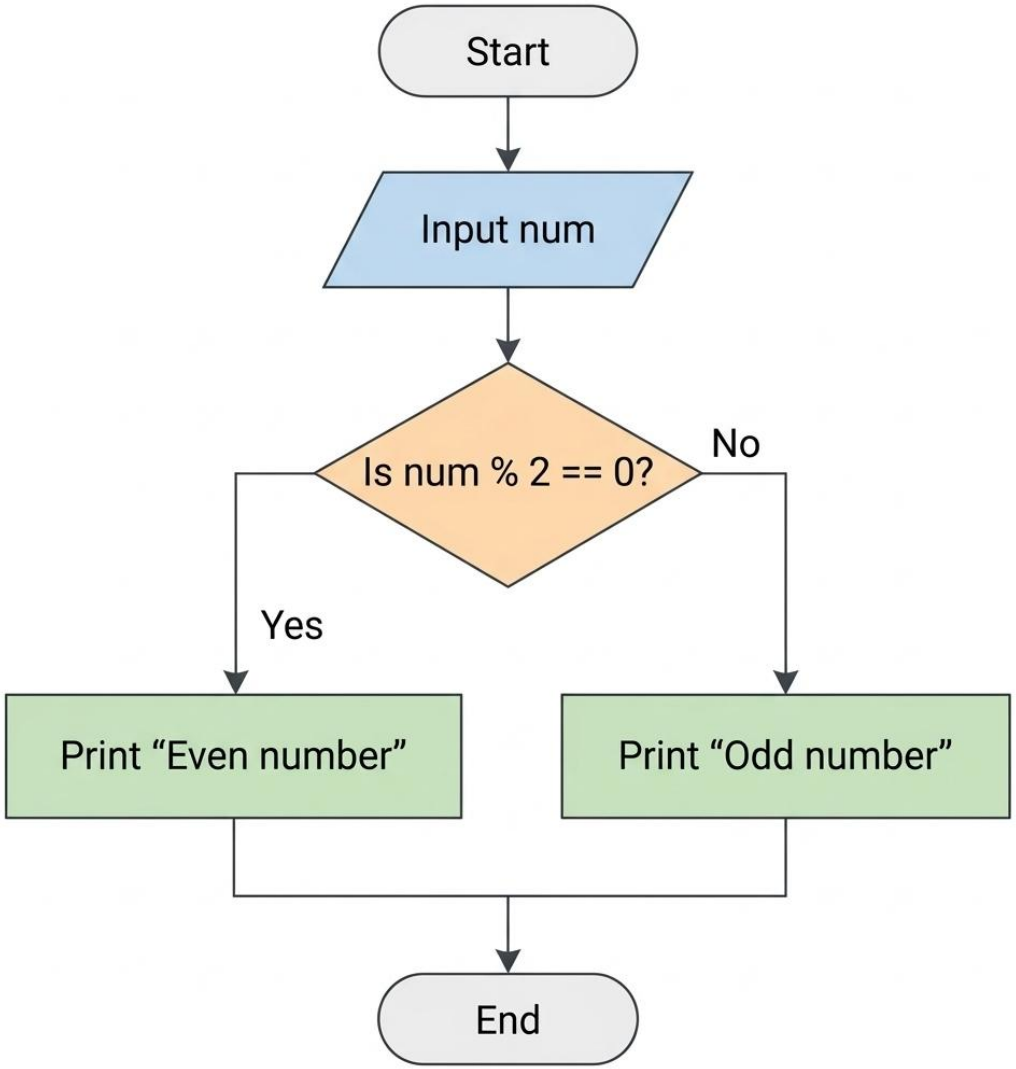
int main(void)
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 == 0)
        printf("Even number");
    else
        printf("Odd number");

    return 0;
}
```

SAMPLE OUTPUT

```
Enter a number: 7
Odd number
```



KEY IDEA

- % calculates the remainder.
- Remainder 0 means even.

Program 4 — if–else: Greater of two

```c
#include <stdio.h>

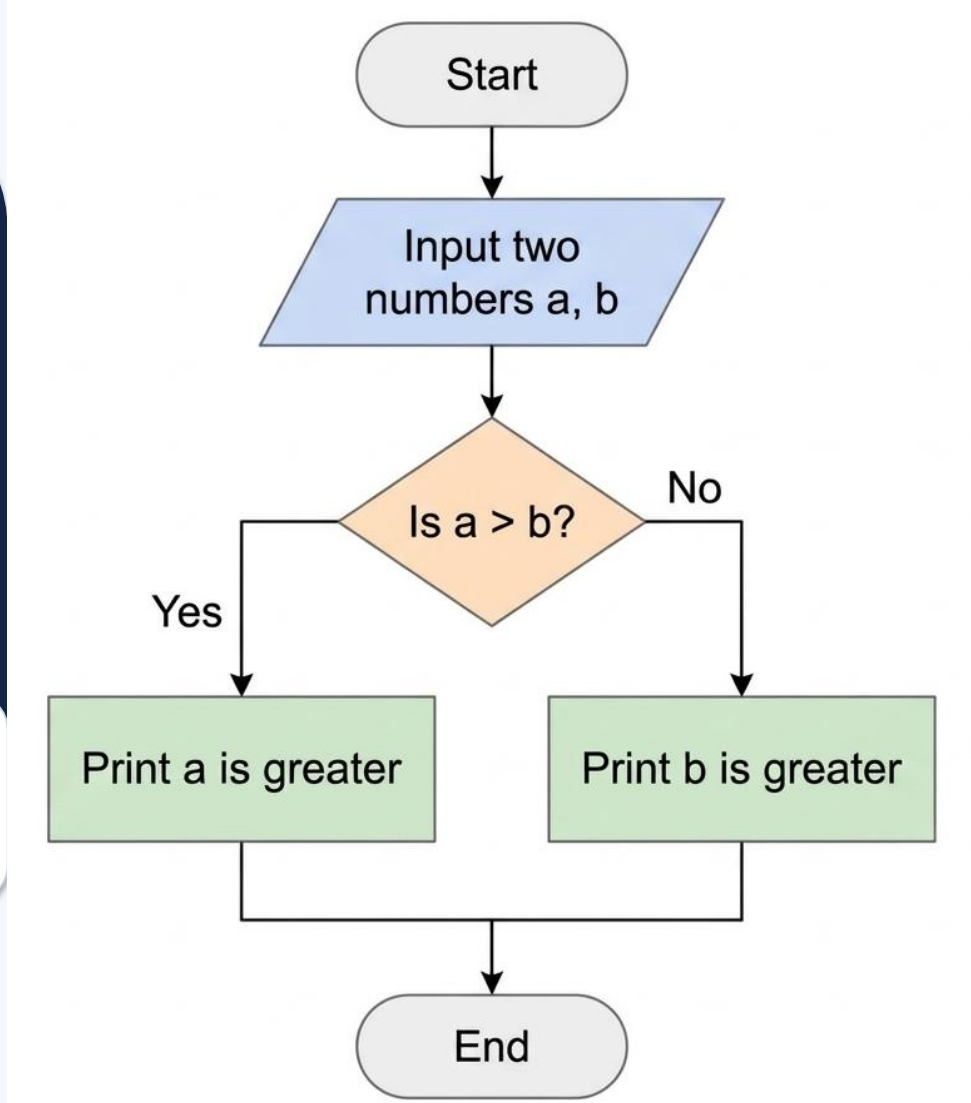
int main(void)
{
    int a, b;
    printf("Enter two different numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("%d is greater", a);
    else
        printf("%d is greater", b);

    return 0;
}
```

SAMPLE OUTPUT

```
Enter two different numbers: 12 5
12 is greater
```



KEY IDEA

- Compares a and b.
- Assumes inputs are different.

Nested if–else — flowchart and algorithm

ALGORITHM

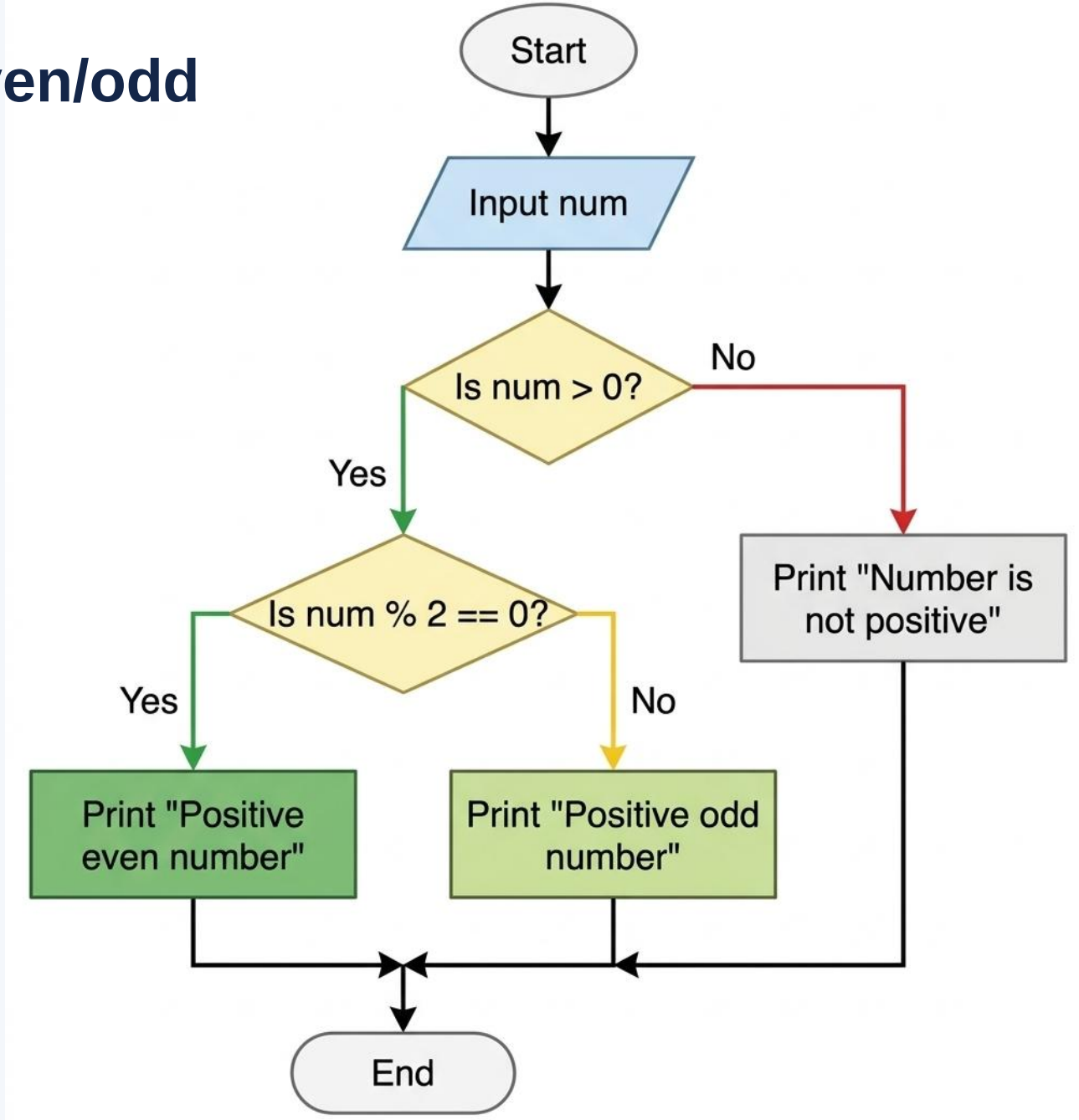
1. Start.
2. Read the value.
3. Check the outer condition.
4. If required, check the inner condition.
5. Display the appropriate result and stop.

Program 5 — Nested if: Positive even/odd

```c
#include <stdio.h>

int main(void)
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0)
    {
        if (num % 2 == 0)
            printf("Positive even number");
        else
            printf("Positive odd number");
    }
    else
    {
        printf("Number is not positive");
    }
    return 0;
}
```



KEY IDEA

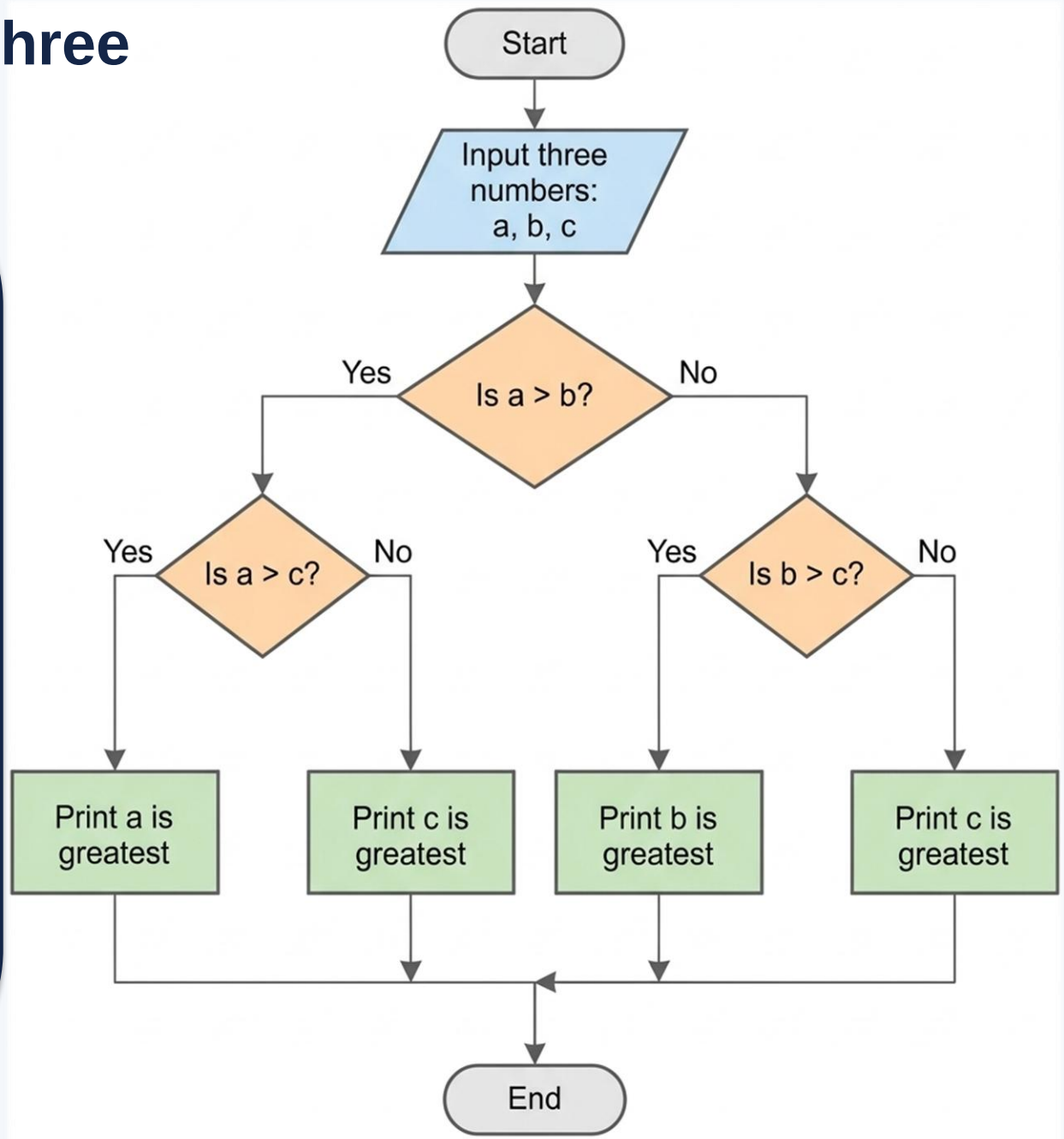
- First checks positivity
- then checks even/odd.

Program 6 — Nested if: Greatest of three

```c
#include <stdio.h>

int main(void)
{
    int a, b, c;
    printf("Enter three different numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b)
    {
        if (a > c)
            printf("%d is greatest", a);
        else
            printf("%d is greatest", c);
    }
    else
    {
        if (b > c)
            printf("%d is greatest", b);
        else
            printf("%d is greatest", c);
    }
    return 0;
}
```

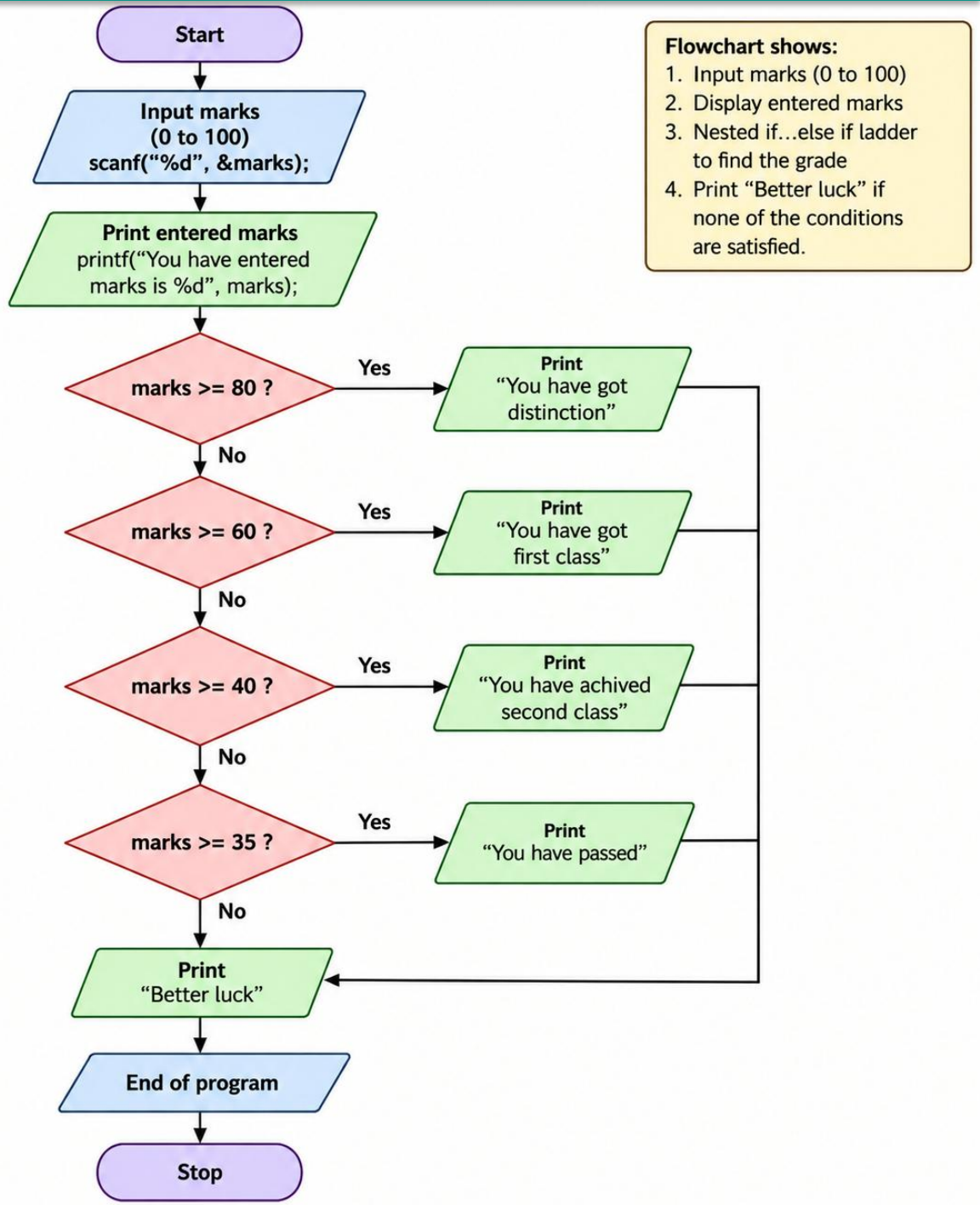


else-if ladder — flowchart and algorithm

Write a program to display Students grade by taking marks input

ALGORITHM

1. Start.
2. Read the value.
3. Check conditions in order.
4. Execute the first matching branch.
5. If no condition matches, execute else.



Program 7 — else-if ladder: Grade

```
int main()
{
      int marks;
      printf("Enter your marks here 0 to 100 \t:");
      scanf("%d",&marks);
      printf("You have entered marks is %d",marks);
      //first condition starts
      if (marks>=80)
      {
            printf("\nYou have got distinction");
      }
      else if (marks>=60)
      {
            printf("\nYou have got first class");
      }
      else if (marks>=40)
      {
            printf("\nYou have achived second class");
      }
      else if (marks>=35)
      {
            printf("\nYou have passed");
      }
      printf("\n Better luck");
      return 0;
}
```

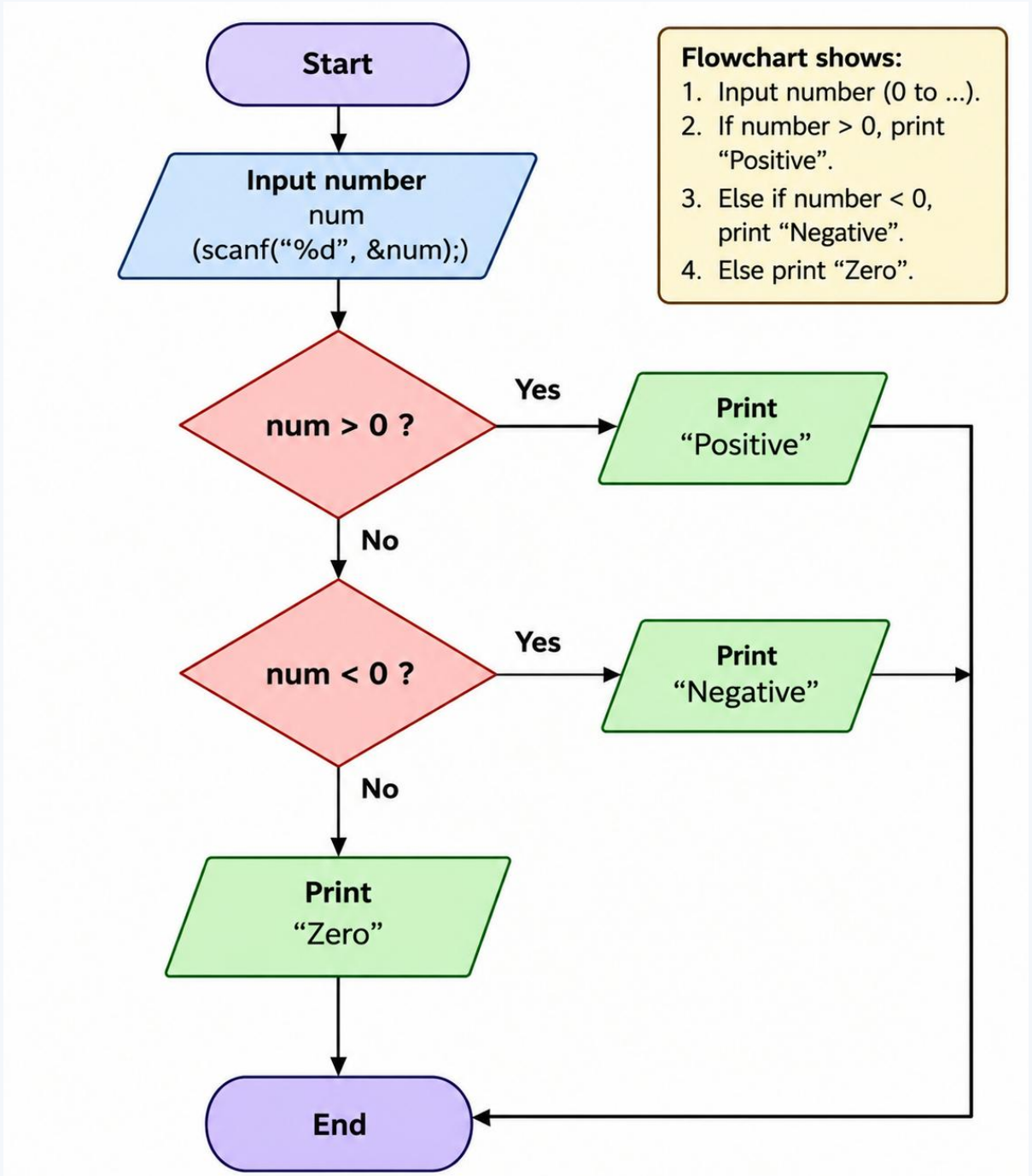
Program 8 — else-if ladder: Sign of number

```c
#include <stdio.h>

int main(void)
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0)
        printf("Positive");
    else if (num < 0)
        printf("Negative");
    else
        printf("Zero");

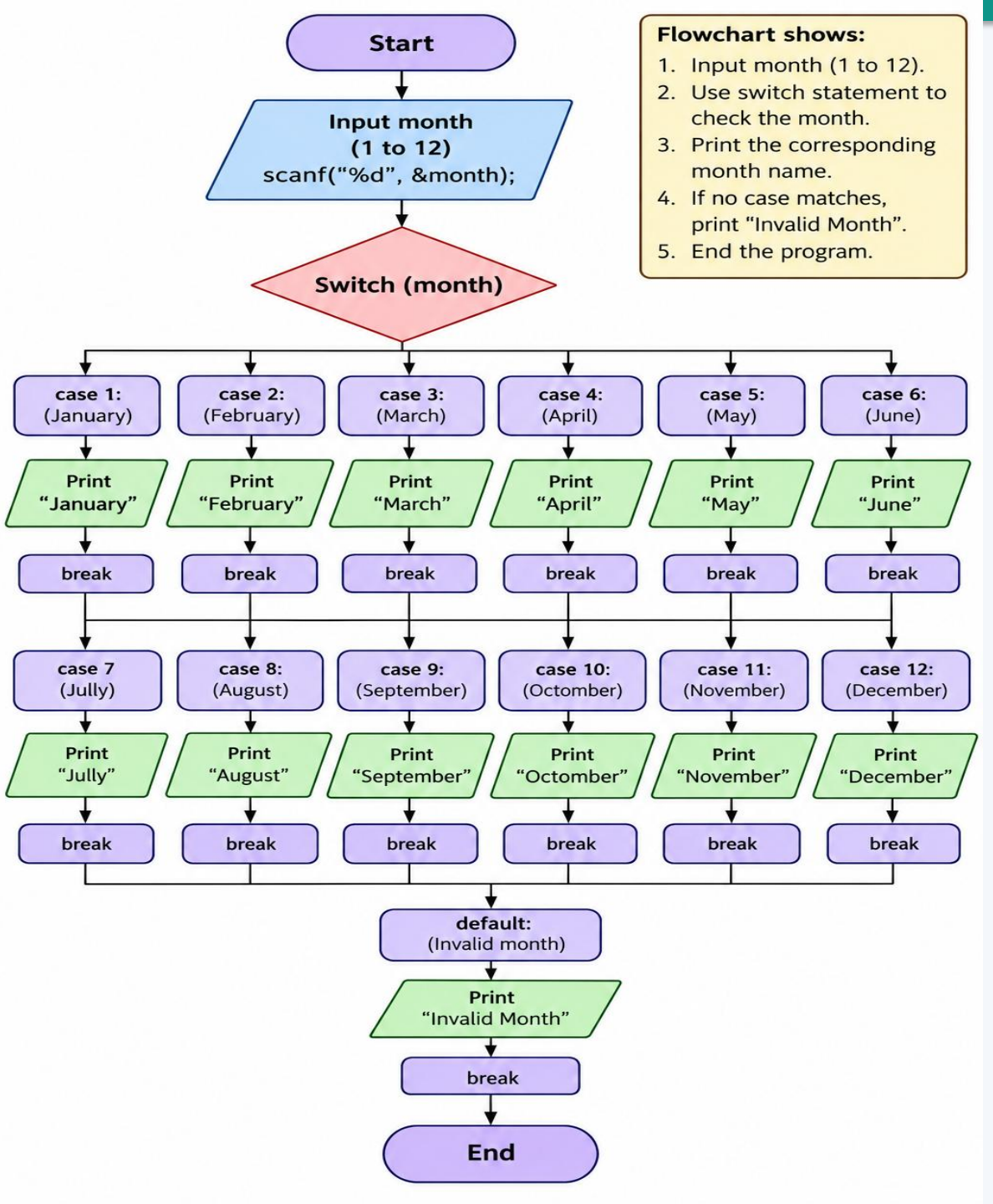
    return 0;
}
```



switch — flowchart and algorithm

ALGORITHM

1. Start.
2. Read the user's choice.
3. Evaluate the switch expression.
4. Run the matching case; use break to exit.
5. Use default if no case matches.



Program 9 — switch: Month number

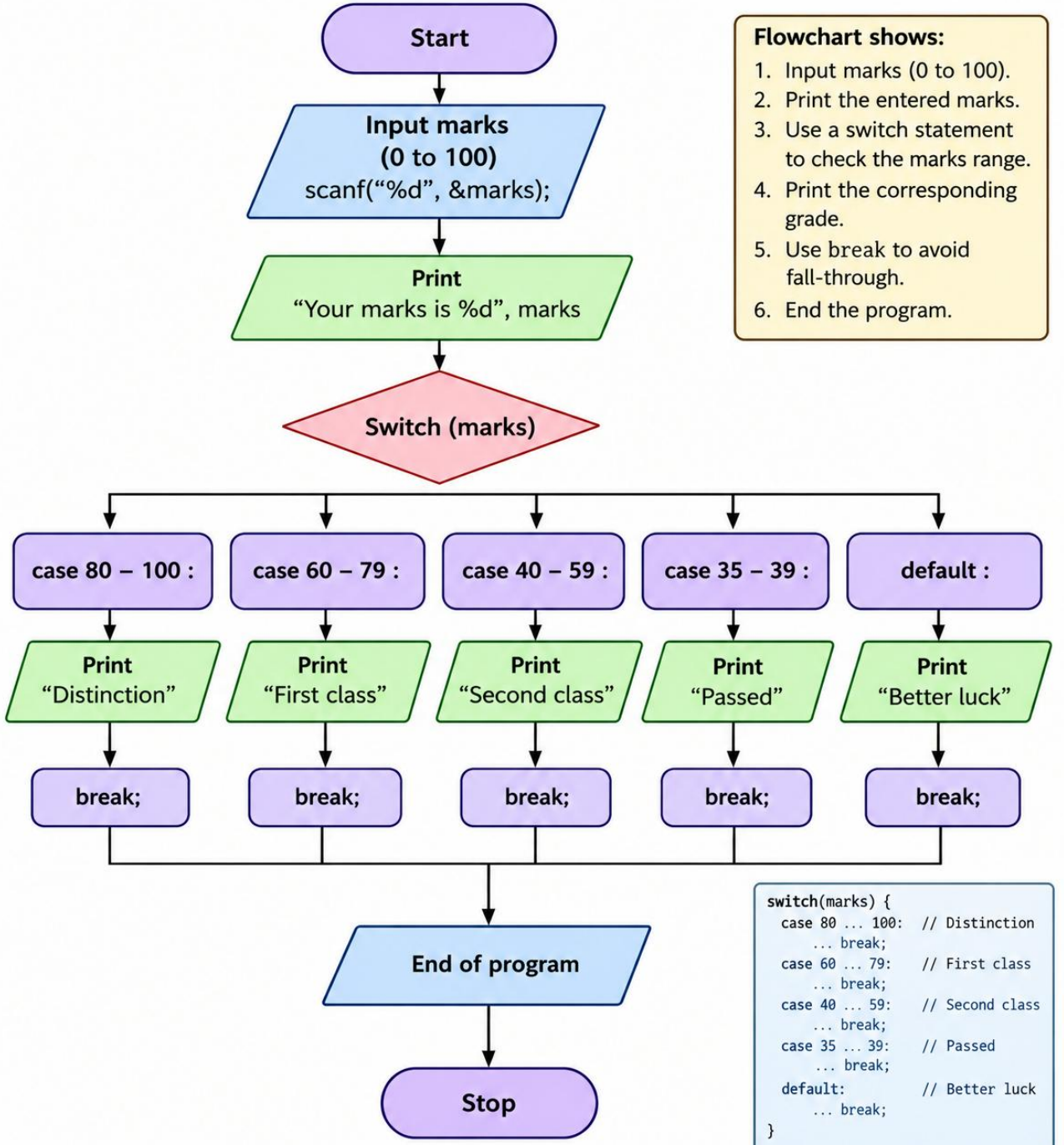
```
#include<stdio.h>

int main()
{
      int month;
      printf("Enter here your selected month in 1 to 12\t:");
      scanf("%d",&month);

      switch (month)
      {
            case 1:printf("January");break;
            case 2:printf("February");break;
            case 3:printf("March");break;
            case 4:printf("April");break;
            case 5:printf("May");break;
            case 6:printf("June");break;
            case 7:printf("Jully");break;
            case 8:printf("August");break;
            case 9:printf("September");break;
            case 10:printf("Octomber");break;
            case 11:printf("November");break;
            case 12:printf("December");break;
            default: printf("Invalid Month");
      }

}
```

Use Switch statement to -Write a program to display Students grade by taking marks input



Program 10 — switch: Mini calculator

```c
#include<stdio.h>

int main()
{
      int a,b;
      char op;
      printf("Enter first Number\t:");
      scanf("%d",&a);
      printf("Enter Second No\t:");
      scanf("%d",&b);
      printf("Select Which operation to perform(+,- or *)");
      scanf(" %c",&op);

      switch(op)
      {
            case '+':printf("Result = %d",a+b);break;
            case '-':printf("Result = %d",a-b);break;
            case '*':printf("Result = %d",a*b);break;
            default : printf("Invalid Input");

      }
      return 0;
}
```

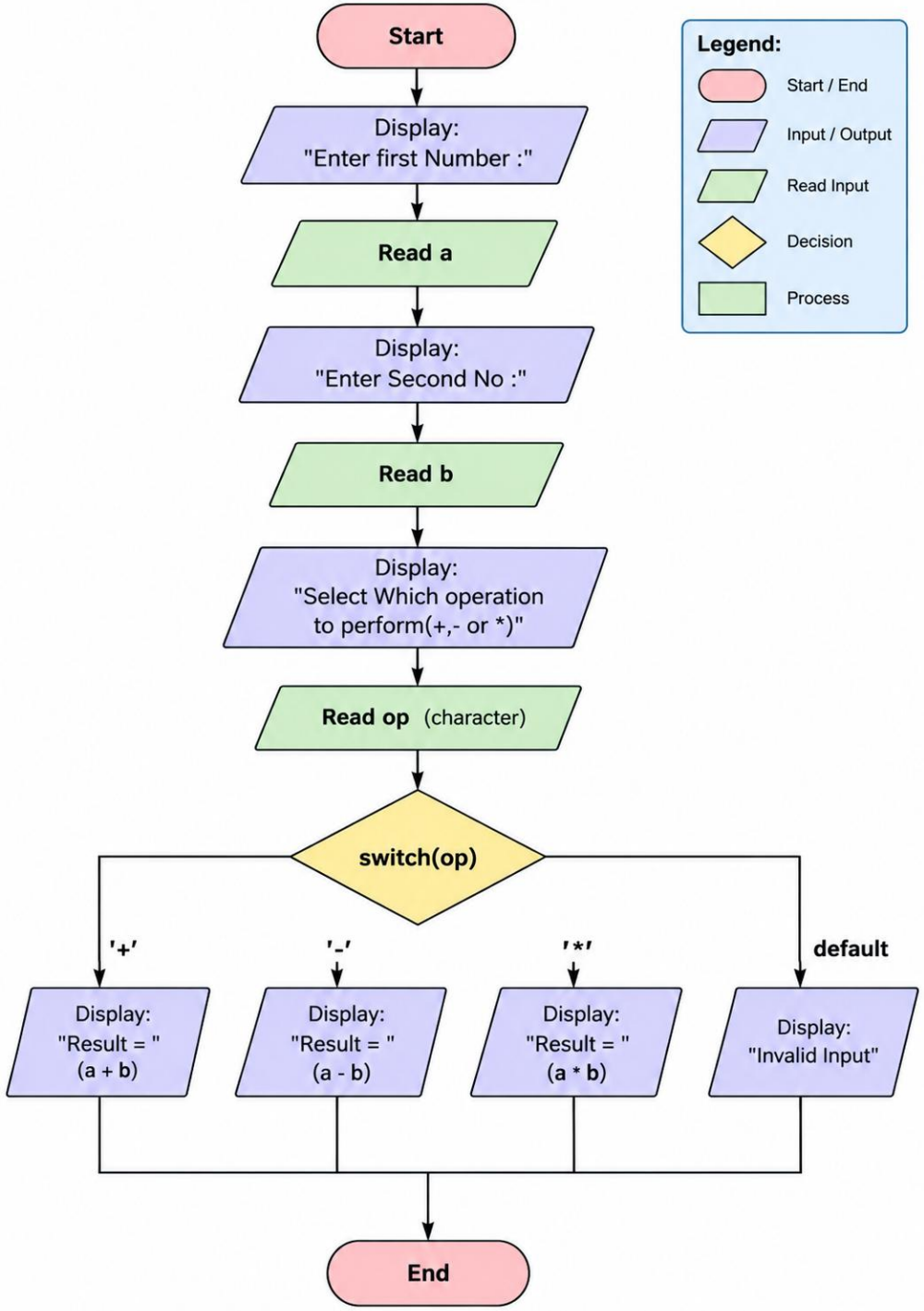
KEY IDEA

- Space before %c skips whitespace.
- Supports +, - and *.

SAMPLE OUTPUT

```
Enter First integers: 8
Enter Second integers: 8
  Enter operator (+, -, *): *
  Result = 24
```

- Start the program.
- Display a message and input the first number a.
- Display a message and input the second number b.
- Ask the user to enter an operator (+, -, or *).
- Check the operator using switch(op).
- If the operator is +, display a + b.
- If the operator is -, display a - b.
- If the operator is *, display a * b.
- For any other operator, display Invalid Input.
- End the program.



while loop — flowchart and algorithm



ALGORITHM

1. Start.
2. Initialize the loop variable.
3. Check the condition.
4. If true, execute the body and update the variable.
5. Repeat; stop when false.

Program 11 — while: Print 1 to 10

```c
#include <stdio.h>

int main(void)
{
    int i = 1;
    while (i <= 10)
    {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

KEY IDEA

- Condition is checked first.
- i++ advances the loop.

1 **Initialization**

`int i = 1;`

The variable `i` starts at 1.

2 **Check the condition**

`while (i <= 10)`

If `i` is less than or equal to 10, execute the loop body.

3 **Print the number**

`printf("%d\n", i);`

`%d` prints the integer, and `\n` moves the cursor to the next line.

4 **Increment the variable**

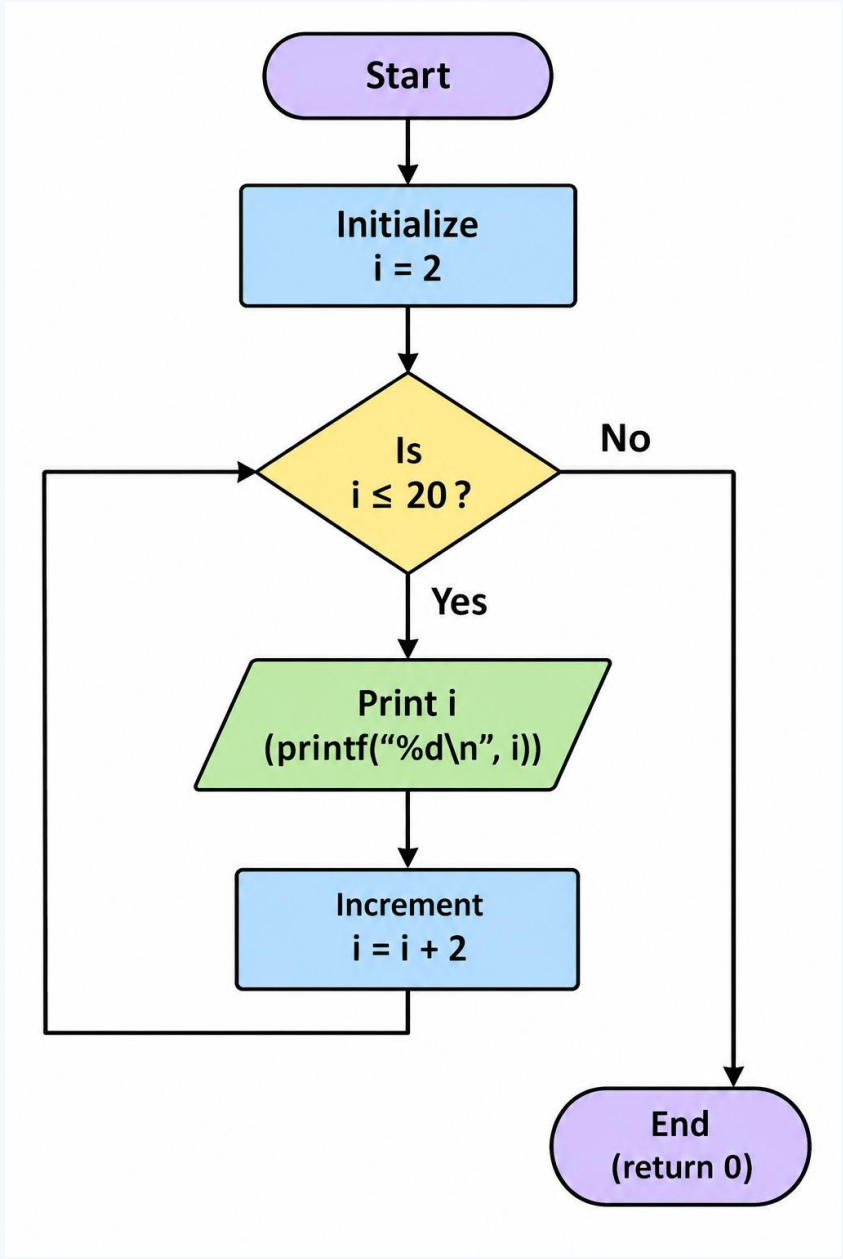
`i++;`

Increases `i` by 1 so the loop can eventually stop.

Program 12 — while: Even numbers to 20

```c
#include <stdio.h>

int main(void)
{
    int i = 2;
    while (i <= 20)
    {
        printf("%d\n", i);
        i = i + 2;
    }
    return 0;
}
```



do–while loop — flowchart and algorithm



ALGORITHM

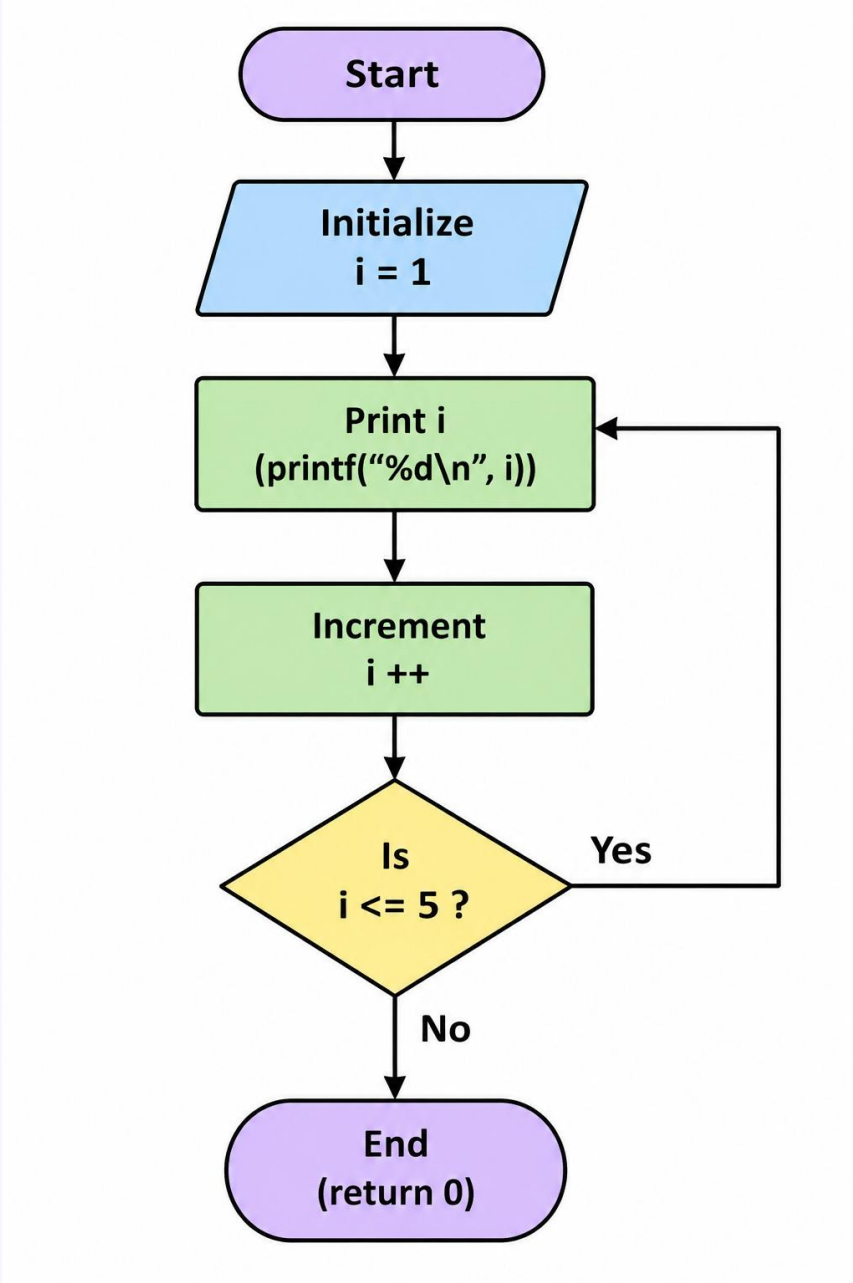
1. Start.
2. Initialize the loop variable.
3. Execute the loop body.
4. Update the variable and check the condition.
5. Repeat if true; otherwise stop.

Program 13 — do–while: Print 1 to 5

```c
#include <stdio.h>

int main(void)
{
    int i = 1;
    do
    {
        printf("%d\n", i);
        i++;
    }
    while (i <= 5);

    return 0;
}
```



Program 14 — do–while: Multiplication table

```c
#include <stdio.h>

int main(void)
{
    int num, i = 1;
    printf("Enter a number: ");
    scanf("%d", &num);

    do
    {
        printf("%d x %d = %d\n", num, i, num * i);
        i++;
    }
    while (i <= 10);

    return 0;
}
```

KEY IDEA

- Prints ten multiples.
- The body executes at least once.

SAMPLE OUTPUT

```
Enter a number: 4
4 x 1 = 4
...
4 x 10 = 40
```

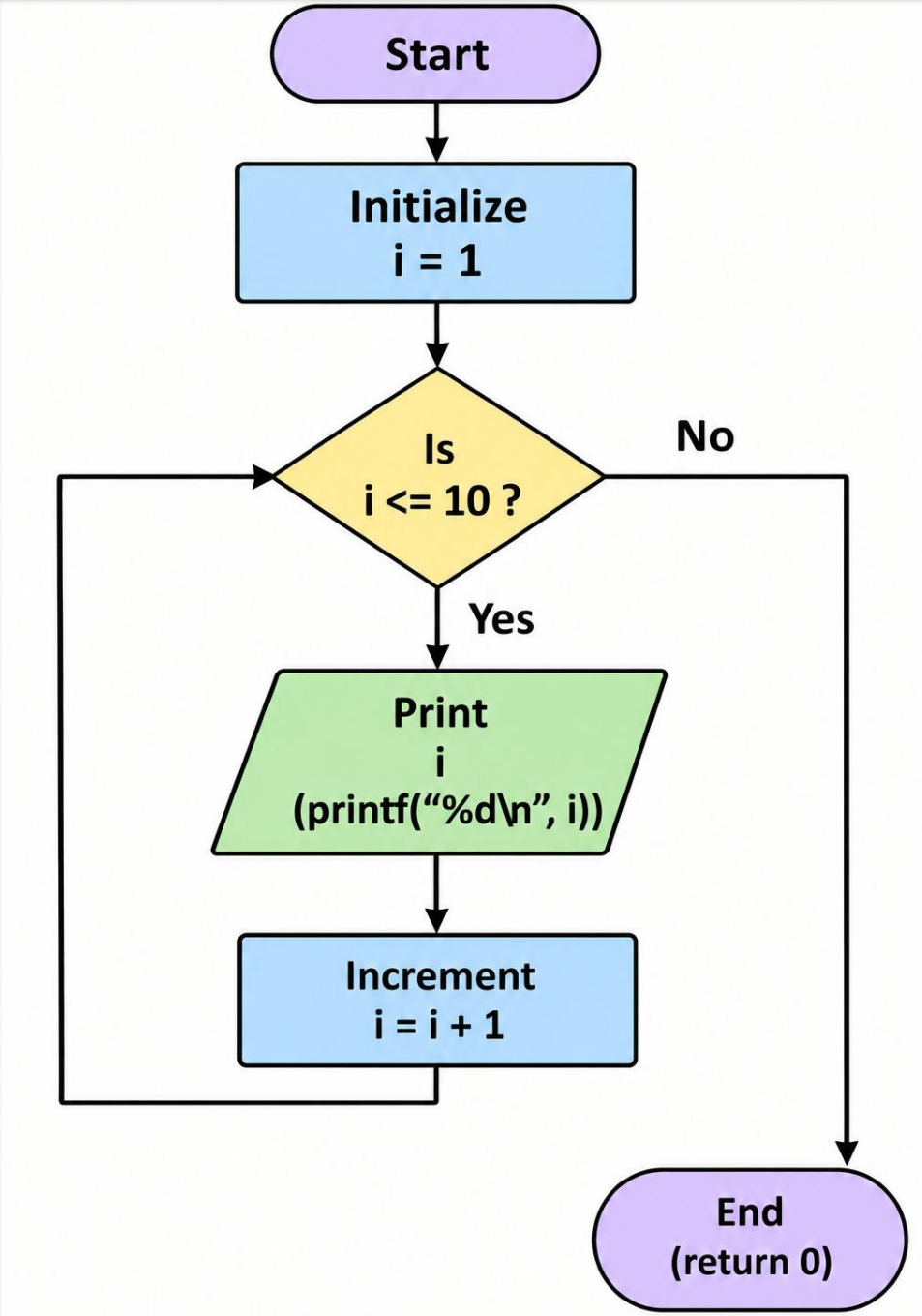
C PROGRAMMING

for loop — flowchart and algorithm

Program 15 — for: Print 1 to 10

```c
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```



KEY IDEA

- Initialization: i = 1.
- Condition: i <= 10; update: i++.

Program 16 — for: Sum of 1 to 10

```c
#include <stdio.h>

int main(void)
{
    int i, sum = 0;
    for (i = 1; i <= 10; i++)
    {
        sum = sum + i;
    }
    printf("Sum = %d", sum);
    return 0;
}
```

KEY IDEA

- sum starts at zero.
- Each pass adds i to sum.

SAMPLE OUTPUT

```
Sum = 55
```

break — flowchart and algorithm

ALGORITHM

1. Start the loop.
2. Check the stopping condition.
3. If true, execute break and exit the loop.
4. Otherwise, execute the remaining body.
5. Continue the loop.

Program 17 — break: Stop at 5

```c
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            break;
        printf("%d\n", i);
    }
    return 0;
}
```

KEY IDEA

- break exits the loop immediately.
- 5 is not printed because break comes first.

SAMPLE OUTPUT

```
1
2
3
4
```

Program 18 — break: First multiple of 7

```c
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 100; i++)
    {
        if (i % 7 == 0)
        {
            printf("First number divisible by 7 = %d", i);
            break;
        }
    }
    return 0;
}
```

KEY IDEA

- The first positive multiple is 7.
- break ends the search after printing.

SAMPLE OUTPUT

```
First number divisible by 7 = 7
```

continue — flowchart and algorithm

ALGORITHM

1. Start the loop.
2. Check the skip condition.
3. If true, skip the remaining body.
4. Otherwise, execute the statements.
5. Proceed to the next iteration.

Program 19 — continue: Skip 5

```c
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            continue;
        printf("%d\n", i);
    }
    return 0;
}
```

KEY IDEA

- Skips printing 5.
- In a for loop, update still runs.

SAMPLE OUTPUT

```
1
2
3
4
6
7
8
9
10
```

Program 20 — continue: Skip negative inputs

```c
#include <stdio.h>

int main(void)
{
    int i, num;
    for (i = 1; i <= 5; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &num);

        if (num < 0)
            continue;

        printf("Non-negative number = %d\n", num);
    }
    return 0;
}
```

KEY IDEA

- Reads exactly five inputs.
- Negative values are skipped; zero is accepted.

SAMPLE OUTPUT

```
Enter number 1: 4
Non-negative number = 4
Enter number 2: -3
Enter number 3: 0
Non-negative number = 0
...
```

Loop comparison: while, do–while and for

- while: checks the condition before running the body; may run zero times.
- do–while: runs the body first; executes at least once.
- for: places initialization, condition and update together in the loop header.
- All three can repeat a task; choose the loop that makes the logic clearest.

break vs continue

- break: exits the nearest loop or switch immediately.
- continue: skips the rest of the current loop iteration.
- In a for loop, continue moves to the update expression and then checks the condition.
- In while and do–while loops, continue proceeds to the condition check.

Student practice worksheet

- 1. Use if to print a message when temperature is above 30.
- 2. Use if–else to check whether a number is divisible by 5.
- 3. Use an else-if ladder to classify marks into four grades.
- 4. Use switch to display a month name from a number.
- 5. Use while to print numbers from 10 down to 1.
- 6. Use do–while to display a menu at least once.
- 7. Use for to calculate factorial of a positive integer.
- 8. Use break to stop when the input is zero.
- 9. Use continue to skip odd numbers from 1 to 20.

- Decisions select which statements execute.
- Loops repeat statements while a condition allows.
- break stops; continue skips one iteration.